

Exercises for Week 13

1. Excessive I/O

Consider the following Python program that prints a short story about a number for the first 100,000 integers:

```

1 for i in range(100_000):
2     # Story by ChatGPT
3     print(
4         f"Once upon a time, {i} felt lonely, being just one among many.
5         "
6         f"Dreaming of distinction, {i} doubled itself, becoming 2*{i}.
7         "
8         f"Still, it wasn't enough, so {i} squared itself, turning into
9         {i}^2. "
10        f" Realizing every transformation retained its essence, "
11        f"{i} learned to cherish its identity, understanding it was
12        unique, "
13        f"infinite, essential-forever {i}, the protagonist of its own "
14        f"mathematical saga."
15    )

```

1. Make a job script that runs the above Python program. Add the `-u` option to Python to run with unbuffered output. You must submit the job to the hpc queue, request 1 core and a CPU model for repeatability. Also, make sure you specify output files with the `-o/-e` options and that the files are under `/work3/02613/dump/`.
2. Make another job script where you manually redirect the output of the Python program to a separate set of files. Again, make sure these files are also under `/work3/02613/dump/`.
Hint: see the slides.
3. Submit both jobs. Check the job resource summaries at the end of the output files. Did the output get redirected as expected? What is the run time difference?

2. Multi-Threaded NumPy

Consider the following Python program, which creates two sets of 100 random 1000 x 1000 matrices and then multiplies them together. At the end, it prints out the time it took.

```

1 from time import perf_counter as time
2 import numpy as np

```

```

3
4 def matmuls(A, B):
5     assert A.shape[0] == B.shape[0], "A and B must hold same number of
        matrices"
6     assert A.shape[2] == B.shape[1], "A and B must hold compatible
        matrices"
7     n = A.shape[0]
8     C = np.empty((n, A.shape[1], B.shape[2]))
9     for i in range(n):
10        C[i] = np.matmul(A[i], B[i])
11    return C
12
13 A = np.random.rand(100, 1000, 1000)
14 B = np.random.rand(100, 1000, 1000)
15 t0 = time()
16 C = matmuls(A, B)
17 t1 = time()
18 print(t1 - t0)

```

1. Write a job script that runs the Python program. You must submit the job to the hpc queue, request 1 core and a CPU model for repeatability. Submit the job. What is the run time?
2. Change the number of cores to 8 cores and submit again. Did the run time change?
3. **Autolab** Change the job script to enable NumPy multi threading with 8 threads (one for each requested core). Submit again. Did the run time change?
Hint: see the slides.
4. Change the Python program such that the `matmuls` function is parallelized with multi-threading using a `ThreadPool` from the `multiprocessing` module. Submit again. What is the run time now? Why can we use multi-threading for this program instead of using *multi-processing*?
Hint: see the slides from week 5.
5. Change your job script to disable NumPy multi-threading (i.e., remove what you added in exercise 2.3). Submit the job one last time. What is the run time now?